Computational Neuroscience — Week 8

Learning from Supervision and Rewards

Neurons as Classifiers, and Reinforcement Learning

Week 7 learned without a teacher. This week a teacher appears. Lecture 1 takes the idealised neuron from week 1, gives it a learning rule, shows exactly what it cannot do, and fixes that with extra layers — arriving at backpropagation, and so at the origin of most of modern machine learning. Lecture 2 drops the teacher for a reward signal, derives temporal difference learning, and finds its prediction error apparently encoded by dopamine neurons. Lecture 3 turns prediction into action with the actor–critic algorithm, and closes the course. Green boxes give the plain-words version of each idea before the mathematics; rust boxes flag the places where the obvious reading is the wrong one.

Contents

I Neurons as Classifiers **2**

1 The Classification Problem **2**

2 The Perceptron **2**

3 The Perceptron Learning Rule **3**

4 What Perceptrons Cannot Do **4**

5 Continuous Outputs: Sigmoid Units **6**

6 Learning Multilayer Networks **6**

7 Backing Up a Truck **8**

8 Where This Leaves Us **9**

II Reinforcement Learning — Predicting Rewards **9**

9 Learning Without a Teacher's Answers **10**

10 Pavlov's Dog **10**

11 A Neuron That Predicts Reward **10**

12 Temporal Difference Learning **11**

13 Does It Work? **12**

14 Dopamine **14**

15 Where This Leaves Us **15**

III Reinforcement Learning — Time for Action **15**

16 From Prediction to Action 16

17 The Rat in the Barn 16

18 Values Solve the Action Problem 17

19 Actor–Critic Learning 18

20 Does It Find the Optimal Policy? 19

21 The Basal Ganglia 20

22 Reinforcement Learning in Practice 20

23 Where This Leaves Us 20

Part I

Neurons as Classifiers

Lecture 1 — the perceptron, its learning rule, the XOR problem, sigmoid units, and backpropagation.

1. The Classification Problem

[Lecture slide omitted from the public copy.] Images as points in a high-dimensional space, labelled $+1$ for faces and -1 for everything else. Classification means finding a surface that separates them.

The idea in plain words

Treat each image as a *point*. An image of a million pixels is one point in a million-dimensional space, and a collection of images is a cloud of such points. If faces land in one region and non-faces in another, then classifying a new image is a geometric question: which side of the dividing surface does it fall on?
The two-dimensional pictures throughout this lecture are the same problem drawn at a size the page can hold.

Watch out — the labels are the new ingredient

Everything in week 7 learned from the inputs alone. Here every input arrives with a **label** v_d — the answer, supplied by a teacher.
That is what **supervised** means, and it is a strong assumption. Someone had to look at every training image and say "face" or "not face". The lecture is candid that this is why big labelled datasets drove the recent rise of machine learning.

2. The Perceptron

[Lecture slide omitted from the public copy.] The idealised neuron from week 1, now with a name.

The perceptron

Sum the weighted inputs, compare with a threshold, output a spike or not:

$$v \;=\; \Theta\!\left(\sum_i w_i u_i - \mu\right), \qquad \Theta(x) = \begin{cases} +1 & x > 0 \\ -1 & x \le 0 \end{cases}$$

with inputs $u_i \in \{+1, -1\}$ denoting spike or no spike.
Introduced by **Rosenblatt** (1958), building on **McCulloch and Pitts** (1943).

The idea in plain words

Nothing here is new except the name. This is the week 1 cartoon of a neuron — sum your inputs, fire if the total clears threshold — written as one line of algebra.
What is new is asking what that cartoon can *compute*, and how it could *learn*.

What a perceptron does

[Lecture slide omitted from the public copy.] Setting the weighted sum equal to the threshold defines a hyperplane, and the perceptron reports which side you are on.

Set the argument of Θ to zero:

$$\sum_i w_i u_i - \mu \;=\; 0$$

The idea in plain words

Every point where the weighted sum *exceeds* μ lies on one side of that hyperplane and gets output $+1$; everything on the other side gets -1.
So a perceptron performs **linear classification**: linear because the boundary it draws is always flat. The weights w_i set the hyperplane's orientation, and the threshold μ sets how far it sits from the origin.

3. The Perceptron Learning Rule

[Lecture slide omitted from the public copy.] Adjust the weights and the threshold in proportion to the output error.

The perceptron learning rule *the **supervised rule***

With desired output v_d and actual output v, the error is $(v_d - v)$:

$$\Delta w_i = \epsilon\,(v_d - v)\,u_i, \qquad \Delta\mu = -\epsilon\,(v_d - v)$$

The idea in plain words

Read it one case at a time, for a positive input $u_i = +1$:

- **Error positive** means $v_d = +1$ but $v = -1$: the neuron should have fired and did not. The rule *increases* w_i, raising the weighted sum towards the threshold.
- **Error negative** means $v_d = -1$ but $v = +1$: it fired when it should not have. The rule

decreases w_i.

For a negative input the signs reverse, and the rule still does the right thing. When the output is already correct the error is zero and nothing changes at all.

Watch out — the threshold moves the other way

Note the minus sign on $\Delta\mu$. A positive error *raises* the weights but *lowers* the threshold. That is not an inconsistency — both changes push the weighted sum further above μ, which is exactly what a positive error is asking for. Raising the sum and lowering the bar are the same request expressed twice.

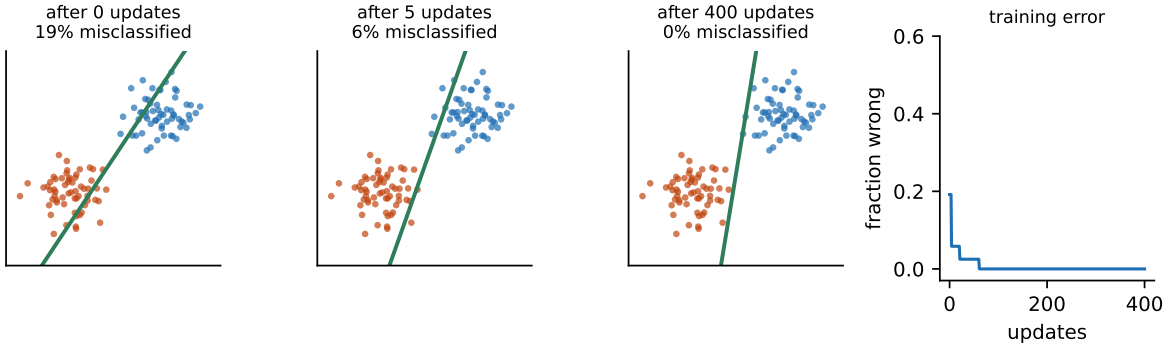


Figure 1: *The rule running on separable data. The boundary starts in the wrong place, is roughly right within five updates, and reaches zero training error.*

The idea in plain words

Compare this with the rules of week 7. Hebbian learning multiplied u by the neuron's *own* output v. This multiplies u by the *error* $(v_d - v)$ — the gap between what the neuron did and what it was told to do.

That single substitution is the whole difference between unsupervised and supervised learning, and it is why this rule needs a teacher while Oja's rule did not.

4. What Perceptrons Cannot Do

[Lecture slide omitted from the public copy.] The exclusive-or function, and its four points in input space.

The idea in plain words

XOR outputs +1 only when the two inputs *differ*. Plotting the four cases, the two +1 points sit at opposite corners of a square, and so do the two −1 points.

No straight line can separate opposite corners from the other pair. A perceptron therefore cannot learn XOR — not because the learning rule is poor, but because *no setting of the weights exists* that would work.

Linear separability

A perceptron can classify a dataset only if some hyperplane separates the two classes. Such data are called **linearly separable**. XOR is not.

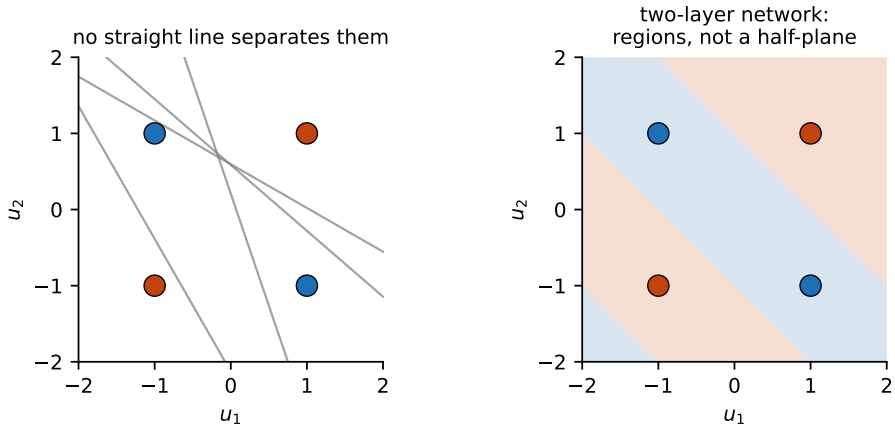


Figure 2: *Left: boundaries from six training runs, none of which separates the classes. Over 200 restarts the best a single perceptron ever achieved was one point in four wrong. Right: the two-layer network's decision regions — a band rather than a half-plane.*

Watch out — a failure of representation, not of training

This distinction matters. The perceptron learning rule is not stuck in a local minimum and would not be rescued by a better optimiser, a smaller learning rate, or more data.
Running 200 restarts from random initial weights, the best error ever reached was 25% — exactly one of the four points, every time. The hypothesis class simply does not contain a solution.

Multilayer perceptrons

[Lecture slide omitted from the public copy.] A two-layer perceptron that computes XOR.

The idea in plain words

Stack them. A hidden unit computes its own half-plane, the output unit combines the hidden units' verdicts, and the composite boundary need not be flat.
In the lecture's network, the hidden unit has weights $(1, 1)$ with threshold $\mu = -1$, so it fires unless both inputs are -1. The output unit takes $(-1, 2, -1)$ from (u_1, h, u_2) with $\mu = 1$.

The lecture invites you to check it. Substituting all four inputs:

u_1	u_2	hidden h	output v	XOR
-1	-1	-1	-1	-1
-1	$+1$	$+1$	$+1$	$+1$
$+1$	-1	$+1$	$+1$	$+1$
$+1$	$+1$	$+1$	-1	-1

All four agree.

The idea in plain words

The trick is visible in the table. The hidden unit distinguishes "both inputs are -1" from everything else, and the output unit then only has to separate "both are $+1$" from the rest

— two easy linear questions whose combination is the hard non-linear one.

5. Continuous Outputs: Sigmoid Units

For **regression** rather than classification — mapping road images to steering angles, say — outputs of ± 1 will not do.

The sigmoid

$$g(a) = \frac{1}{1 + e^{-\beta a}}$$

It maps $(-\infty, +\infty)$ onto $(0, 1)$, and β controls the slope.

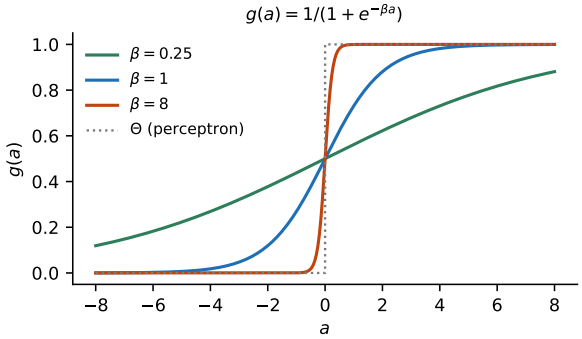


Figure 3: *The sigmoid at three values of β, with the perceptron's step function for comparison.*

The idea in plain words

The output can be read as a **normalised firing rate**: 0 for silent, 1 for the cell's maximum rate. A neuron peaking at 100 Hz has its rate divided by 100 to land in this range.
β interpolates between the two regimes seen so far. Measuring the width of the 10%–90% transition, $\beta = 0.25$ spans 17.6 units of input and is effectively linear; $\beta = 8$ spans 0.55 and is effectively the perceptron's step. Large β recovers Θ; small β recovers a linear unit.

Watch out — smoothness is the point, not realism

The sigmoid is not adopted because it fits firing rates better than a threshold. It is adopted because it is **differentiable**.
Θ has zero derivative everywhere it is defined and none at all at the threshold, so gradients carry no information about how to improve. Everything in the rest of this lecture requires a derivative, and that requirement is what chooses the function.

6. Learning Multilayer Networks

[Lecture slide omitted from the public copy.] A three-layer sigmoid network, the error function, and the two learning rules.

Three-layer sigmoid network *the **forward pass***

$$x_j = g\left(\sum_k w_{jk} u_k\right), \qquad v_i = g\left(\sum_j W_{ij} x_j\right)$$

with w the input-to-hidden weights and W the hidden-to-output weights.

The idea in plain words

Using *many* hidden layers gives **deep networks**, which learn progressively more complex features with depth. That is the whole of modern deep learning, and the lecture points at it in passing — everything needed to understand it is on this slide.

The error to be minimised is the squared output error, summed over output neurons:

$$E(W, w) \;=\; \tfrac{1}{2} \sum_i (d_i - v_i)^2$$

The delta rule

For the output weights, differentiating directly:

$$\Delta W_{ij} \;=\; -\epsilon \frac{\partial E}{\partial W_{ij}} \;=\; \epsilon\,(d_i - v_i)\, g'\left(\sum_j W_{ij} x_j\right) x_j$$

The idea in plain words

This is the **delta rule**, named for the δ that traditionally denotes the error $(d_i - v_i)$.
Its shape is worth noticing: error $\times$ slope $\times$ input. It is the perceptron rule with the hard ± 1 error replaced by a graded one, and a factor g' that scales the step by how responsive the unit currently is.

Backpropagation

The hidden weights are harder: E is defined in terms of the *output* activities, but w_{jk} sits two layers away. The chain rule bridges the gap:

$$\frac{\partial E}{\partial w_{jk}} \;=\; \frac{\partial E}{\partial x_j} \cdot \frac{\partial x_j}{\partial w_{jk}}$$

The idea in plain words

Both factors are computable — the first from the expression for E, the second from the expression for x_j — and substituting them gives the **backpropagation** learning rule.
The name describes the flow. Errors are measured at the output and propagated *backwards* through the network to reach the hidden weights. With more hidden layers the chain rule simply extends, carrying errors down through all of them.

Watch out — backpropagation's biological plausibility is genuinely doubtful

The rule requires each hidden synapse to know the error at distant output neurons, weighted by all the synapses in between. There is no established mechanism by which a real synapse could obtain that information.
Contrast the rules of week 7, which used only $\mathbf{u}$, v and $\mathbf{w}$ — quantities physically present at the synapse. This is a powerful algorithm that came from thinking about neurons, not a claim about how neurons learn.

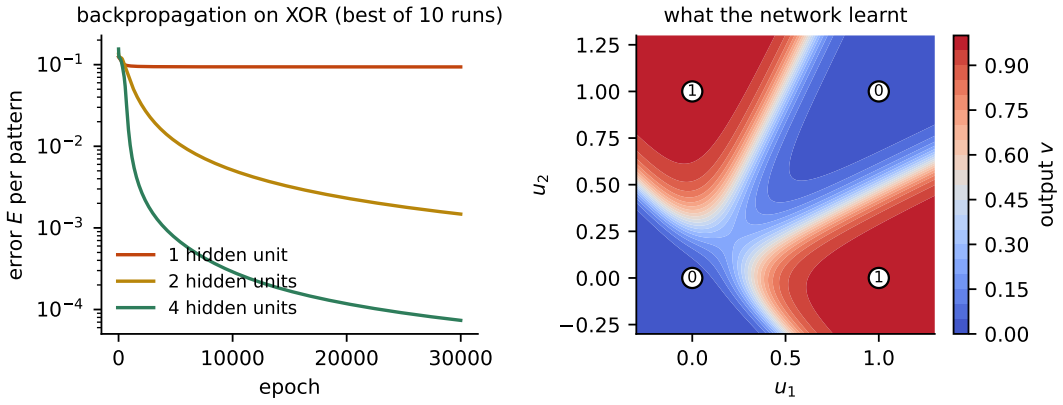


Figure 4: *Backpropagation on XOR with sigmoid units. Left: error against epoch for three network sizes, best of ten runs each. Right: the function a four-unit network learns.*

Watch out — capacity and optimisation are different failures

Training ten runs of each size for 30,000 epochs separates two things that both look like "it didn't work":

hidden units	best final error	runs reaching $< 10^{-3}$
1	9.4×10^{-2}	0/10
2	1.5×10^{-3}	0/10
4	7.4×10^{-5}	10/10

With one hidden unit the error flatlines and never improves however long you train: the network *cannot* represent XOR. With two — the minimum that can — most runs still stall in local minima. With four, every run succeeds.
Extra units beyond the minimum do not add representational power here. They make the error surface easier to descend, which is a different kind of help entirely.

7. Backing Up a Truck

[Lecture slide omitted from the public copy.] The demonstration: a network learns to reverse a simulated truck into a loading dock.

The idea in plain words

The network takes the truck's position x, y and orientation θ, and outputs a steering angle. Training data come from a human reversing the simulated truck, providing the desired steering angle for each configuration.

Early in training it drives erratically; after some thousands of passes over the human data it approaches the dock; eventually it arrives. This is **regression** in exactly the sense set up above — continuous inputs to a continuous output, learnt by imitating a teacher.

Watch out — the network can only imitate

Supervised learning here means copying a human's steering angles. The network has no notion that parking is a *goal*, and no way to discover a manoeuvre the human never demonstrated. Its ceiling is the teacher.
Removing that ceiling — learning from a reward signal rather than from worked examples — is what the next lecture is about, and it is why reinforcement learning exists as a separate subject.

8. Where This Leaves Us

- **Classification is geometry.** Images are points in a high-dimensional space, and classifying means choosing which side of a surface a point falls on.
- **A perceptron draws a hyperplane**, because setting a weighted sum equal to a threshold is a linear equation. The weights orient it; the threshold positions it.
- **Its learning rule is driven by error** rather than by the neuron's own activity — the one substitution that separates supervised from unsupervised learning.
- **XOR is impossible for a single perceptron**, and this is a limit of what it can represent, not of how it is trained.
- **Layers fix that**, by composing several flat boundaries into a curved one.
- **Sigmoid units give continuous outputs** and, more importantly, derivatives — which is what makes gradient methods available at all.
- **The delta rule handles the output layer; backpropagation carries errors back** to the hidden layers by the chain rule. It works well and is hard to defend biologically.
- **Supervised learning imitates a teacher** and cannot exceed one.

To add from lectures: Next lecture: predicting rewards, and how the brain might do it — reinforcement learning.

Part II

Reinforcement Learning — Predicting Rewards

Lecture 2 — classical conditioning, temporal difference learning, and dopamine as a prediction error.

9. Learning Without a Teacher's Answers

[Lecture slide omitted from the public copy.] The reinforcement learning setting: an agent in a state, taking actions, receiving rewards from an environment.

The idea in plain words

Animals learning to find food in a maze, ride a bike or play the piano are not handed the correct answer at each moment. They act, and something good or bad follows. Learning proceeds by trial and error against **rewards** and punishments.
The formal setting: at time t the agent is in state $\mathbf{u}_t$ — for a rat in a barn, its location — and may receive a scalar reward r_t, positive for food or negative for an encounter with a cat. The problem is to choose actions maximising the **total expected future reward**.

Watch out — a weaker signal than supervision, in two ways

Last lecture's teacher supplied the desired output for every input. A reward says only *how good* the outcome was, not *what should have been done* — and it may arrive long after the action that earned it.
Both gaps have to be closed before anything can be learnt, and this lecture addresses the second one only. Selecting actions is deferred to lecture 3; here the sole task is *predicting* reward.

10. Pavlov's Dog

[Lecture slide omitted from the public copy.] Classical conditioning: bell, then food; eventually the bell alone produces salivation.

The idea in plain words

Pavlov rang a bell and followed it with food, many times over. Eventually the dog salivated at the bell alone.
The interpretation is the one that matters here: the **conditioned stimulus** (the bell) has come to *predict* a future reward (the food). The dog's brain has learnt a prediction, and it acts on the prediction rather than on the reward itself.

The problem, formalised

Given many trials, each of length T steps, with stimulus $u(t)$ and reward $r(t)$ at each step, we want a neuron whose output predicts the expected total future reward from time t onwards:

$$v(t) \;\approx\; \left\langle \sum_{\tau=0}^{T-t} r(t+\tau) \right\rangle_{\text{trials}}$$

11. A Neuron That Predicts Reward

[Lecture slide omitted from the public copy.] A tapped delay line feeding all past stimuli into a weighted sum.

The prediction *the linear filter*

$$v(t) \;=\; \sum_{\tau=0}^{t} w(\tau)\, u(t-\tau)$$

The idea in plain words

This is a **discrete linear filter** — exactly the object from week 2, where it described how a neuron's response depends on a stimulus history. Linear filtering strikes again, now predicting reward rather than encoding a stimulus.
The **tapped delay line** is what makes it possible: the network is given access to the stimulus at every past time step, so $w(\tau)$ can express "a bell τ steps ago is worth this much".

The obvious next step is to minimise the squared error between prediction and actual total future reward,

$$\left[\sum_{\tau=0}^{T-t} r(t+\tau) - v(t)\right]^2$$

by gradient descent and the delta rule, as in lecture 1.

Watch out — the future rewards are not available

The sum runs over $r(t), r(t+1), r(t+2), \ldots$ — and at time t only $r(t)$ has happened. The error function cannot be evaluated, so the gradient cannot be computed.
This is the real obstacle, and it is not a technicality. Any system learning from delayed reward faces it.

12. Temporal Difference Learning

The resolution goes back to **Richard Bellman** and **dynamic programming**: rewrite the error recursively so the unavailable terms disappear.

The idea in plain words

Split the troublesome sum into the reward now and everything after:

$$\sum_{\tau=0}^{T-t} r(t+\tau) \;=\; r(t) + \sum_{\tau=0}^{T-t-1} r(t+1+\tau)$$

and then make the key substitution: the second term is the total future reward from $t+1$ onwards — which is precisely what the network already estimates as $v(t+1)$.

$$\sum_{\tau=0}^{T-t} r(t+\tau) \;\longrightarrow\; r(t) + v(t+1)$$

Temporal difference learning *the* ***Sutton and Barto, 1980s***

Minimising $[\,r(t) + v(t+1) - v(t)\,]^2$ by gradient descent gives

$$\Delta w(\tau) \;=\; \epsilon \underbrace{\big[r(t) + v(t+1) - v(t)\big]}_{\delta}\, u(t-\tau)$$

The idea in plain words

The name comes from the bracket, which contains a *difference between predictions at two adjacent times*, $v(t+1) - v(t)$.
δ is the **prediction error**: how much better things turned out than the network expected. If reward plus the revised estimate exceeds the old estimate, $\delta > 0$ and the weights rise.

The idea in plain words

Worth noticing what $w(\tau)$ means here. It is indexed by *delay since the stimulus*, not by which input line it belongs to. After learning, $w(\tau)$ answers "how much future reward does a bell τ steps ago still predict?"

Watch out — the network is trained on its own output

This is a genuinely strange move, and worth being uneasy about. The unavailable future rewards have been replaced by the network's own guess at them — so the target is partly manufactured by the thing being trained.
It is called **bootstrapping**, and it is not obviously legitimate. The reason it works is that $r(t)$ is real: every update injects one step of genuine information, and that grain of truth propagates backwards one step per trial until the whole prediction is anchored to actual reward.

13. Does It Work?

Taking the lecture's example — stimulus at $t = 100$, reward of total size 2 at $t = 200$ — and running the rule.

[Lecture slide omitted from the public copy.] The result from the lecture.

The idea in plain words

Before learning, v is flat zero and δ spikes at $t = 200$: the reward arrives as a complete surprise, and δ is simply the reward.
After learning, v rises to exactly 2.000 at the moment of the stimulus and holds there until the reward — the network has learnt that a bell is worth two units of food. And δ at the reward is now 0.0000: the reward is fully expected, so it carries no news.

Watch out — the error moves to $t = 99$, not $t = 100$

After learning the error peaks at $t = 99$ with value exactly $+2$, one step *before* the stimulus arrives.
That is not an off-by-one bug. Since $\delta(t) = r(t) + v(t+1) - v(t)$, the error registers wherever

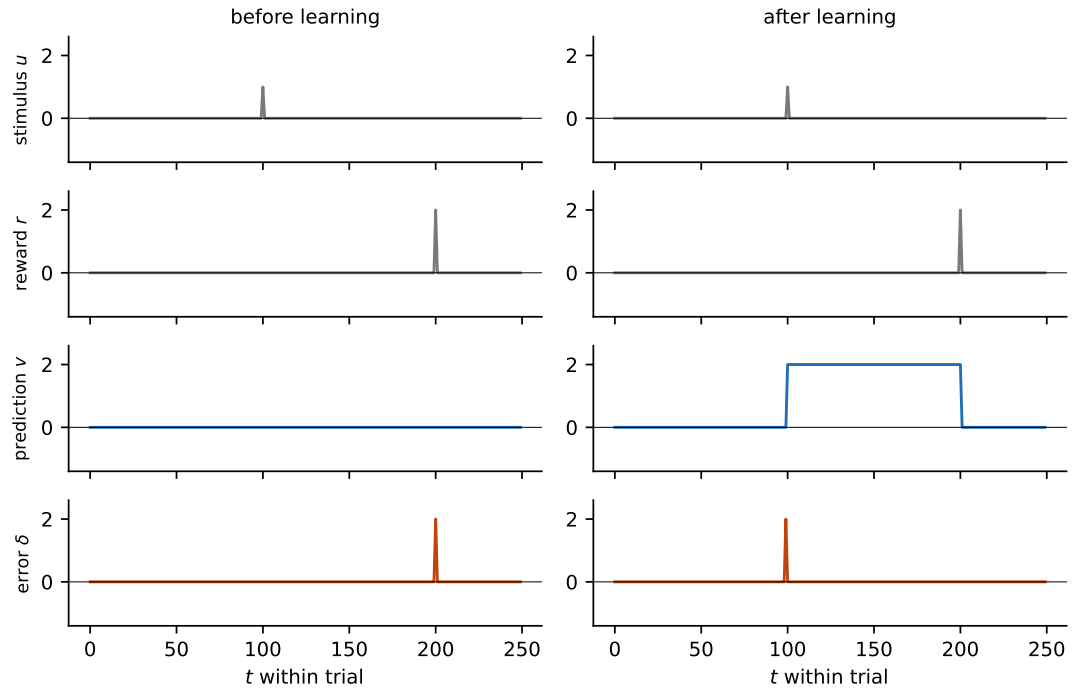


Figure 5: *The same simulation. Stimulus and reward are identical before and after; the prediction and the error are not.*

v changes: $\delta(99) = v(100) - v(99) = 2 - 0$. By $t = 100$ both $v(100)$ and $v(101)$ are already 2, so δ is back to zero. The surprise is the transition, not the state.

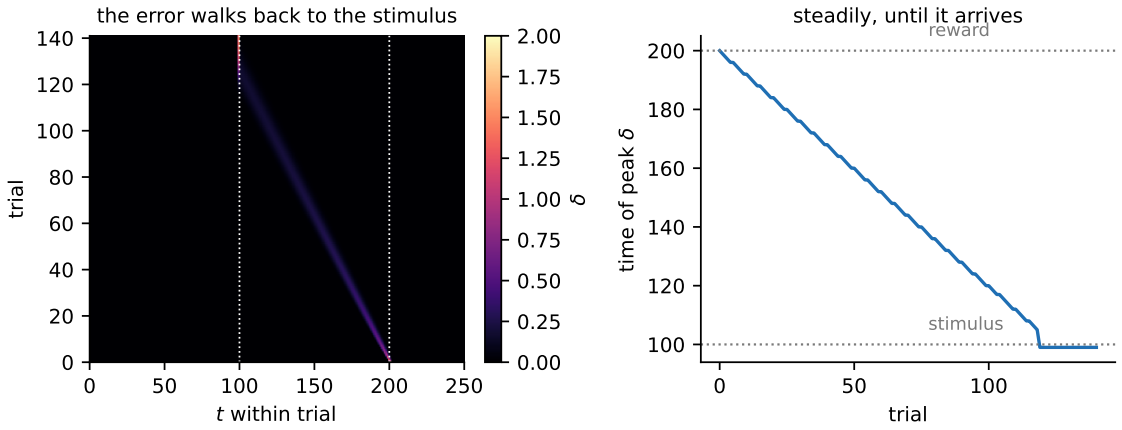


Figure 6: *The prediction error across 140 trials. It starts at the reward and retreats steadily to the stimulus — 101 steps in 119 trials at $\epsilon = 0.8$ — then stays there.*

The idea in plain words

This is the mechanism made visible. On trial 1 the error sits at the reward. Each trial, the step just before the current error learns to predict it, so the error moves one notch earlier. The bump crawls backwards until it reaches the earliest reliable predictor — the bell — and stops.

The rate depends on ϵ: at $\epsilon = 0.8$ it advances about 0.85 steps per trial. A smaller learning rate simply takes longer; the destination is the same.

14. Dopamine

[Lecture slide omitted from the public copy.] Recordings from dopaminergic cells in the ventral tegmental area, before and after training.

The idea in plain words

Schultz and colleagues recorded from the **ventral tegmental area** (VTA) of the midbrain in monkeys. VTA cells are **dopaminergic**, and dopamine has long been implicated in reward-based learning and in addiction.
The monkey heard a stimulus, pressed a key, and was rewarded shortly after. Before training, the cells fired strongly at the *reward*. After training, they no longer responded at the reward — they responded at the *stimulus*.

The idea in plain words

Those are the two panels of the simulation above. The claim that follows is one of the most cited results in computational neuroscience: VTA dopamine neurons appear to encode the **reward prediction error** δ, not reward itself.
It explains both observations at once. Before training the reward is unpredicted, so δ is large at reward time. After training $r(t) + v(t+1) - v(t) = 0$ there, while the stimulus now carries the whole surprise.

[Lecture slide omitted from the public copy.] And when an expected reward is withheld.

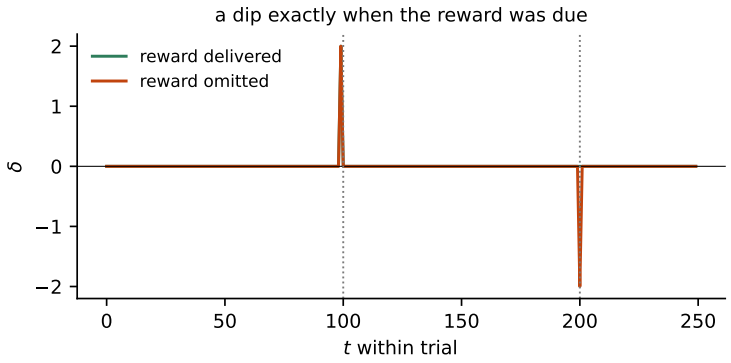


Figure 7: *Simulated: the same trained network, run once with the reward and once without. Identical until $t = 200$, where the omitted case drops to exactly -2.*

The idea in plain words

The model makes a sharp prediction. Omit the expected reward and $r(t) = 0$ while $v(t+1) = 0$ too, so $\delta = -v(t)$ — a *negative* error of exactly the size that was predicted, here -2.000. Schultz and colleagues observed precisely this: VTA cells *dip below baseline* at the moment the reward was due but did not arrive.

Watch out — this is what makes the result convincing

A theory that merely fitted the before-and-after training data would be unremarkable — plenty of accounts could produce a response that moves earlier.
The dip is different. It is a signed, quantitative prediction about what happens in a condition the model was not built to explain, and firing rates have nowhere obvious to go but down. That is a real test, and it passed.

Watch out — encoding a quantity is not computing with it

The evidence says VTA activity *correlates* with δ. It does not establish that the brain implements this particular algorithm, that v is stored in a tapped delay line, or that dopamine is used by downstream synapses the way $\epsilon\,\delta\,u$ is used here.
The delay line especially is a modelling convenience: maintaining a separate weight for every past time step is not a plausible piece of neural machinery.

15. Where This Leaves Us

- **Reward is a weaker signal than supervision** — it says how good an outcome was, not what should have been done, and it arrives late.
- **Classical conditioning is prediction.** The bell comes to stand for the food.
- **The predictor is a linear filter** over a tapped delay line — week 2's machinery pointed at reward.
- **The obvious error function cannot be computed**, because it needs rewards that have not happened yet.
- **Bellman's trick replaces the unavailable future with the network's own estimate**, giving the TD rule with its prediction error $\delta = r(t) + v(t+1) - v(t)$.
- **Bootstrapping works because each update contains one real reward**, and that truth propagates backwards a step per trial.
- **The error migrates from reward to stimulus** and vanishes at the reward once prediction is perfect.
- **Dopamine neurons in VTA behave like** δ, including the dip below baseline when a predicted reward is withheld — a signed prediction the model was not built to produce.

> *To add from lectures: Next lecture: selecting actions to maximise reward, not merely predicting it.*

Part III

Reinforcement Learning — Time for Action

Lecture 3 — policies, state values, the actor–critic algorithm, exploration, and the basal ganglia.

16. From Prediction to Action

The policy

A **policy** π maps states to actions, $\pi(\mathbf{u}) \to a$. The goal is the policy maximising the expected total future reward

$$\left\langle \sum_{\tau=0}^{T-t} r(t+\tau) \right\rangle_{\text{trials}}$$

— the same quantity lecture 2 learned to *predict*, now to be *maximised*.

The idea in plain words

The loop closes here. The agent reads a state $\mathbf{u}_t$, may receive a reward r_t, and now also takes an action a_t, which changes the environment and determines the next state.
Lecture 2 answered "how much reward should I expect?". This one answers "what should I do?".

17. The Rat in the Barn

[Lecture slide omitted from the public copy.] Three decision points, two actions at each, and food at the leaves.

The idea in plain words

A rat in a barn faces three choice points A, B and C, going left or right at each. The rewards sit at the four end locations: from B, left gives 0 and right gives 5; from C, left gives 2 and right gives 0. A leads to B or C.
In the formal language: **states** are locations, **actions** are left and right. Negative rewards would be predators; the lecture is kind and uses none.

Values under a random policy

[Lecture slide omitted from the public copy.] The value of each state if the rat chooses left or right at random.

The value of a state

$v(u)$ is the expected total future reward starting from state u, under whatever policy is being followed.

The idea in plain words

For a uniformly random policy the arithmetic is immediate:

$$v(B) = \tfrac{1}{2}(0) + \tfrac{1}{2}(5) = 2.5$$
$$v(C) = \tfrac{1}{2}(2) + \tfrac{1}{2}(0) = 1$$
$$v(A) = \tfrac{1}{2}v(B) + \tfrac{1}{2}v(C) = 1.75$$

Note how $v(A)$ is built from the values of its successors rather than from rewards directly. That recursion is the same idea as lecture 2's substitution, and it is what makes the problem

tractable.

Learning them online

The rat does not see the tree; it experiences locations one at a time. So learn $v(u)$, represented by a weight $w(u)$, using the TD rule from lecture 2:

$$w(u) \;\leftarrow\; w(u) + \epsilon\big[r(u) + v(u') - v(u)\big]$$

where u' is the state reached by taking an action at u.

[Lecture slide omitted from the public copy.] The lecture's result: jagged individual estimates, with running averages converging on 1.75, 2.5 *and* 1.

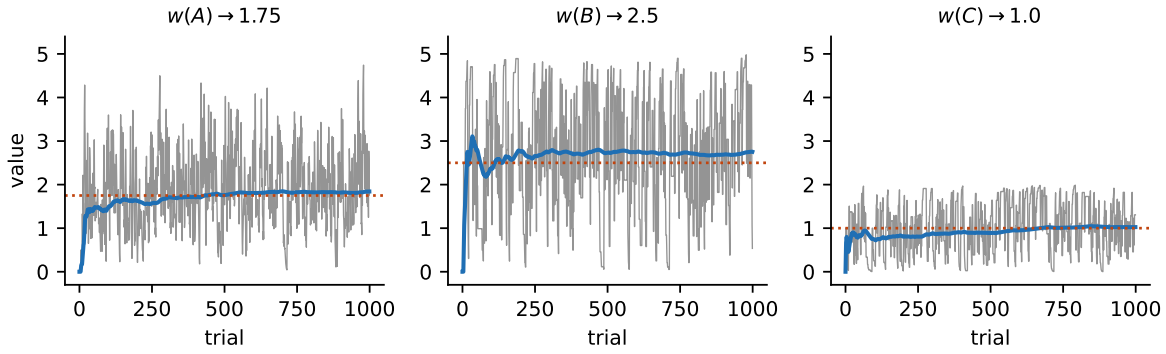


Figure 8: *The same simulation. Grey traces are the raw weights, blue the running average, dotted red the hand-calculated value.*

The idea in plain words

The estimates jump around because $\epsilon = 0.5$ is deliberately large — it learns fast at the cost of a noisy estimate, which is precisely the trade-off.
Averaged over 20 runs the rule recovers $v(A) = 1.766 \pm 0.057$, $v(B) = 2.515 \pm 0.096$ and $v(C) = 1.015 \pm 0.040$ against hand values of 1.75, 2.5 and 1.

18. Values Solve the Action Problem

[Lecture slide omitted from the public copy.] Choosing the action that leads to the higher-valued state.

The idea in plain words

Once the values are known, action selection is trivial: from A, going left leads to B with value 2.5 and going right leads to C with value 1, so go left.
The important move is that **values act as surrogate immediate rewards**. Neither B nor C pays anything on arrival, so a rat comparing immediate rewards would see a tie and have no basis to choose. The value summarises everything good that lies beyond, collapsing the whole future into one number attached to the next step.

Local choices, global optimality

In a **Markov** environment — one where the next state depends only on the current state and action — repeatedly taking the locally best step yields the globally optimal policy. This can be proved rigorously, and is closely related to Bellman's **dynamic programming**.

Watch out — this guarantee is doing a lot of work

"Greedy is optimal" is false for most optimisation problems. It holds here only because the value function already contains the future: a one-step lookahead is secretly a full lookahead, because v was built recursively.
And it needs the Markov property. If the best action depended on how the rat arrived at A, or on something it saw two rooms ago, then a value attached to the location alone could not capture it and the argument collapses.

19. Actor–Critic Learning

[Lecture slide omitted from the public copy.] The two components and their update rules.

Actor and critic

Critic — maintains the value of each state (*policy evaluation*), by the TD rule:

$$w(u) \leftarrow w(u) + \epsilon\big[r(u) + v(u') - v(u)\big]$$

Actor — selects actions and maintains the policy (*policy improvement*). It chooses probabilistically by a **softmax** over state–action values $Q_a(u)$,

$$P(a;u) = \frac{\exp(\beta Q_a(u))}{\sum_b \exp(\beta Q_b(u))}$$

and updates *every* action a' by

$$Q_{a'}(u) \leftarrow Q_{a'}(u) + \epsilon\big[r(u) + v(u') - v(u)\big]\big(\delta_{aa'} - P(a';u)\big)$$

Repeat both steps.

The idea in plain words

$Q_a(u)$ is like a value, but attached to a state *and action* pair rather than a state alone — the worth of doing a here.
Both components are driven by the *same* prediction error $\delta = r(u) + v(u') - v(u)$. One signal, two uses: the critic corrects its estimate of how good the state was, and the actor adjusts how likely it is to repeat what it just did.

Watch out — the bracket is not just a Kronecker delta

The factor is $(\delta_{aa'} - P(a';u))$. For the action actually taken this is $1 - P(a;u) > 0$; for every other action it is $-P(a';u) < 0$.
So a better-than-expected outcome raises the chosen action's Q *and lowers its alternatives*, which keeps the softmax probabilities moving relative to one another rather than all drifting

up together. Subtracting P also shrinks the update for actions already taken often — a built-in brake on reinforcing what is already near-certain.

Exploration versus exploitation

The idea in plain words

Why choose probabilistically at all, rather than simply taking the highest Q?
Because early on the Q values are based on almost no experience and are probably wrong. Committing to them immediately means never discovering that the neglected action was better. This is the **exploration–exploitation dilemma**.
β is the dial. Small β flattens the softmax towards random choice — explore. Large β sharpens it towards always picking the best known action — exploit. Raising β as the estimates improve moves the agent from one regime to the other.

20. Does It Find the Optimal Policy?

[Lecture slide omitted from the public copy.] The lecture's result: probability of going left at each location.

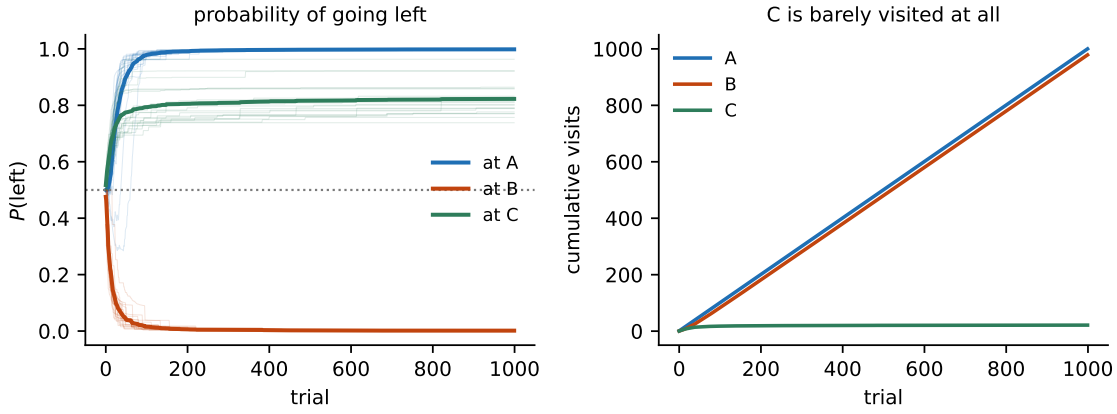


Figure 9: *The same algorithm, averaged over 20 runs with individual runs faint. Left: $P(\text{left})$ at each location. Right: cumulative visits.*

The idea in plain words

The optimal policy is left at A (towards the richer subtree), right at B (the 5), and left at C (the 2). The algorithm finds it: $P(\text{left})$ goes to 0.998 at A and 0.001 at B, settling in a median of 40 and 24 trials respectively, in every one of 20 runs.

Watch out — C is the interesting case, and it mostly fails

C reaches only $P(\text{left}) = 0.823$ on average, and in 17 of 20 runs never settles above 0.9 at all within 1000 trials.
The reason is in the right-hand panel. Once the policy at A commits to going left, the rat stops visiting C: across the runs, A is visited 1000 times and C about 21. A state you have stopped exploring is a state you cannot learn about — and the better your policy gets, the less you learn about the paths you have abandoned.

This is the exploration–exploitation dilemma showing up as an actual failure, not as an abstract concern. Here it is harmless, since C is genuinely the worse branch. It would not be harmless if the rewards behind C were larger than they first appeared.

21. The Basal Ganglia

[Lecture slide omitted from the public copy.] A proposed mapping between actor–critic components and basal ganglia anatomy.

The idea in plain words

Barto and others have suggested that the **basal ganglia** implement something like actor–critic learning. In the mapping, striatal circuits split into an actor and a critic, and the **dopamine** signal (DA, from SNc) plays the role of the TD prediction error — connecting directly to lecture 2's VTA recordings.
A hidden layer appears because mapping state estimates to values and to actions needs more than a single linear layer, as lecture 1 established.

Watch out — the lecture calls this abstract, and means it

This is a high-level correspondence between boxes in a diagram and named anatomical structures, not a demonstration that the basal ganglia compute these equations. Its value is as a starting point for more detailed models.
The honest summary of the whole week: the reward prediction error has real empirical support from the VTA recordings; the architectural mapping around it is a hypothesis.

22. Reinforcement Learning in Practice

[Lecture slide omitted from the public copy.] Autonomous helicopter flight, from Ng, Abbeel and colleagues at Stanford.

The idea in plain words

The closing demonstration: a helicopter performing aerobatic manoeuvres under a learnt policy, using a dynamics model together with a reward function inferred from human demonstrations.
Note the difference from lecture 1's truck, which imitated a human's steering angles directly and could never exceed its teacher. Here the demonstrations are used to infer *what the human was trying to achieve*, and the policy is then optimised against that objective — which is why the result can outperform the demonstrator.

23. Where This Leaves Us

- **A policy maps states to actions**, and the goal is the policy maximising expected total future reward.
- **The value of a state is defined recursively** from the values of its successors, which is what makes it learnable online by the TD rule from lecture 2.

- **Values act as surrogate immediate rewards**, letting an agent choose between actions whose payoffs lie far in the future.
- **Greedy choice is globally optimal in Markov environments** — but only because the value function already encodes the future, and only if the state really is a sufficient summary of the past.
- **Actor–critic splits the job**: the critic evaluates the current policy, the actor improves it, and one prediction error drives both.
- **Probabilistic action selection buys exploration**, with β controlling the trade-off against exploitation.
- **Exploitation starves the abandoned branches.** In simulation the rarely-visited state failed to converge in 17 of 20 runs, having been visited 21 times against another state's 1000.
- **The basal ganglia may implement something like this**, with dopamine as the prediction error — well supported for the error signal, speculative for the architecture.

This is the final lecture of the course.